

Study and familiarization of the basic logic IC's (OR, AND, NOT, NOR, NAND, XOR, XNOR) and Universality of NAND & NOR GATE

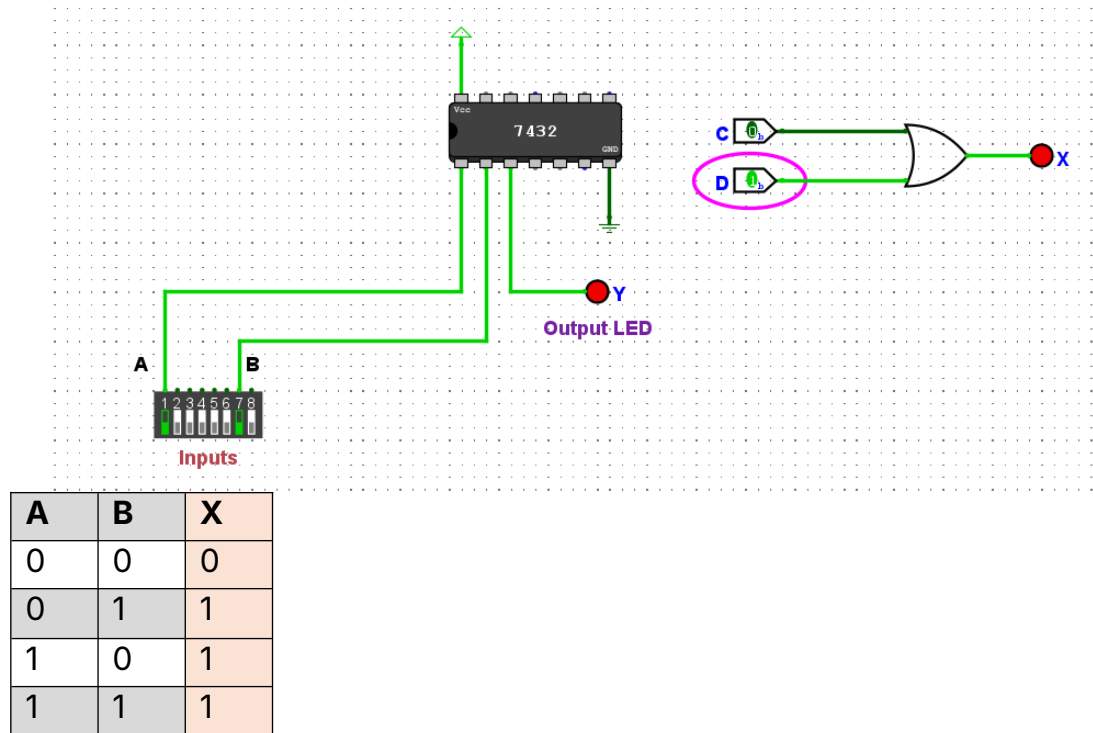


Figure 1: OR Gate (7432) Connect diagram with Truth Table

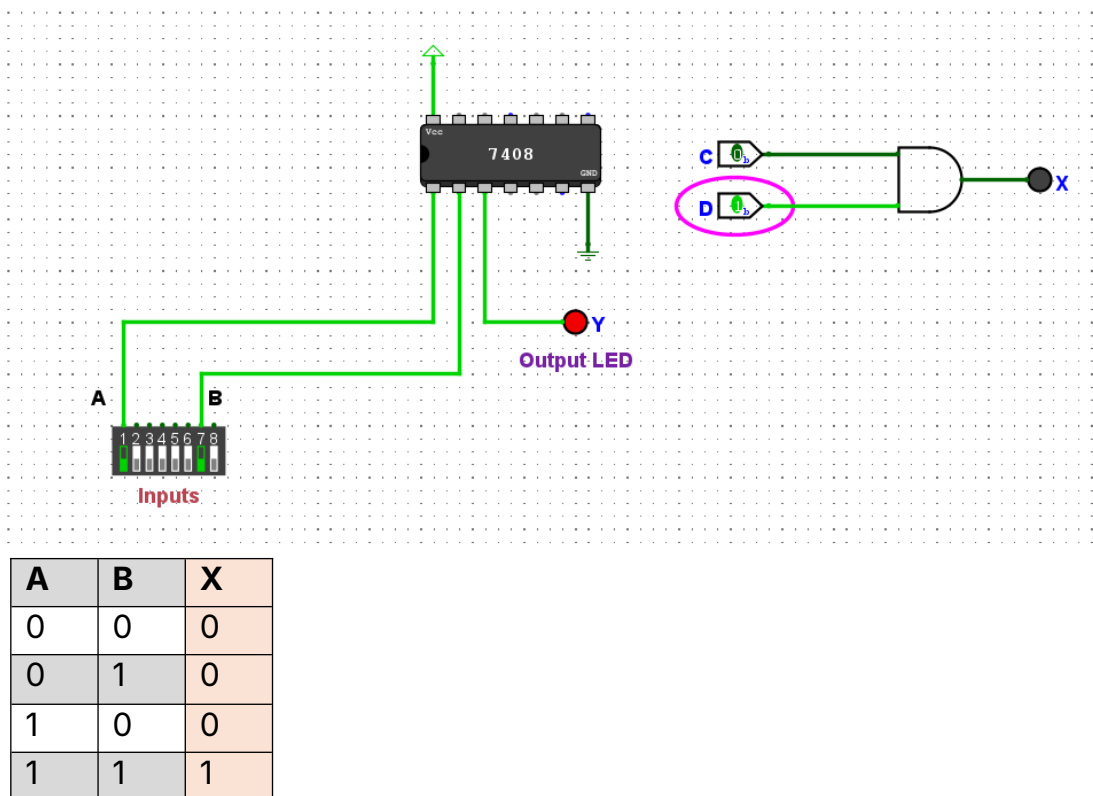
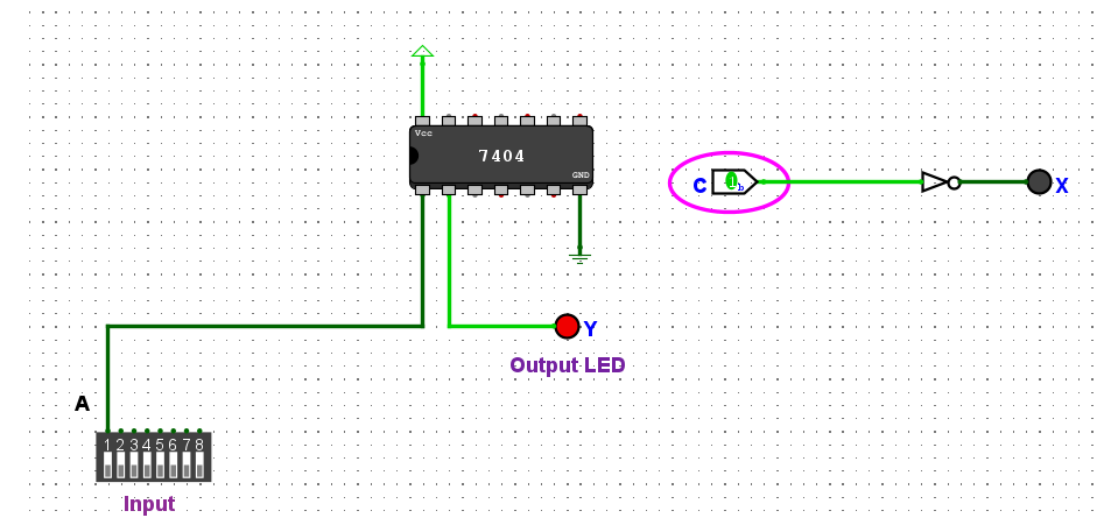
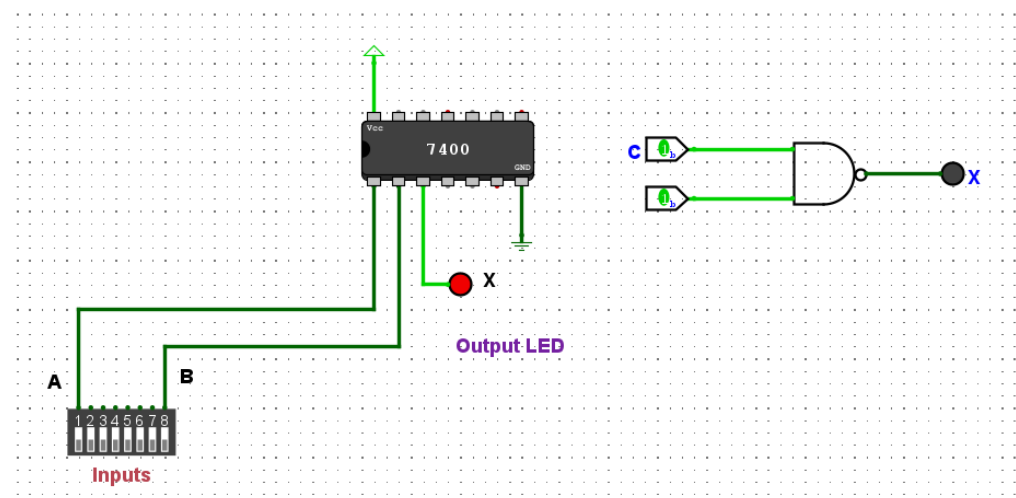


Figure 2: AND Gate (7408) Connect diagram with Truth Table



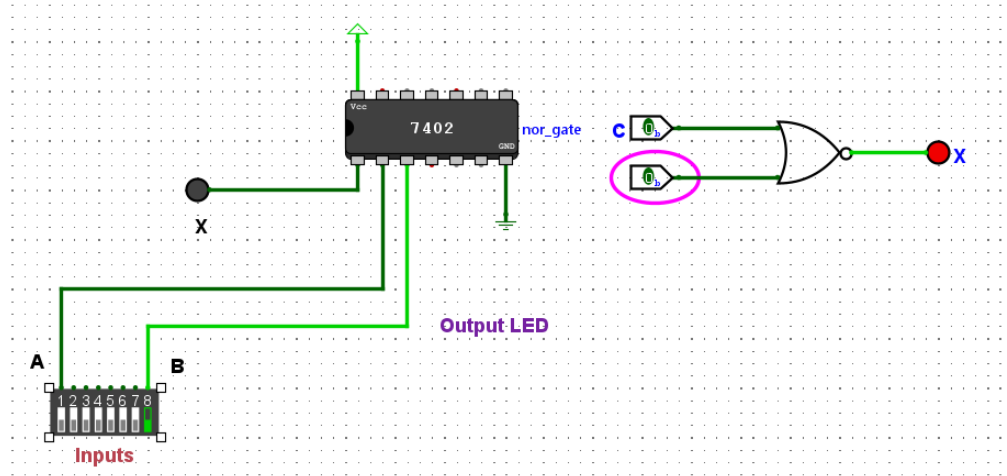
A	X
0	1
1	0

Figure 3: NOT Gate (7404) Connect diagram with Truth Table



A	B	AB	(AB) [']
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Figure 4: NAND Gate (7400) Connect diagram with Truth Table



A	B	A+B	(A+B)'
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

Figure 5: NOR Gate (7402) Connect diagram with Truth Table

A	B	A'	B'	A'B	AB'	A'B+AB'
0	0	1	1	0	0	0
0	1	1	0	1	0	1
1	0	0	1	0	1	1
1	1	0	0	0	0	0

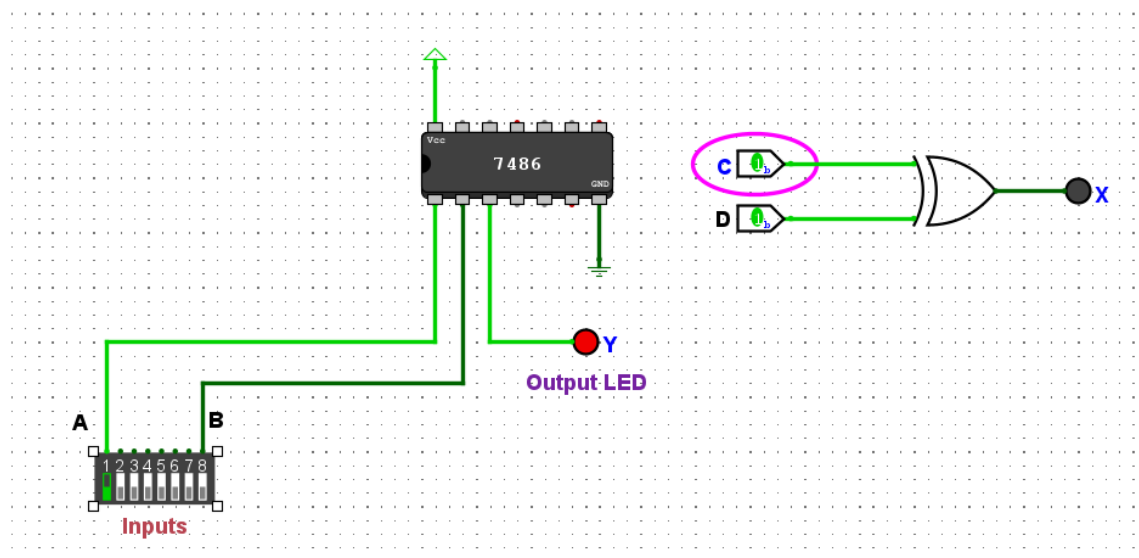


Figure 6: XOR Gate (7486) Connect diagram with Truth Table

A	B	A'	B'	AB	A'B	AB+A'B
0	0	1	1	0	1	1

0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	1	0	0	1	0	0

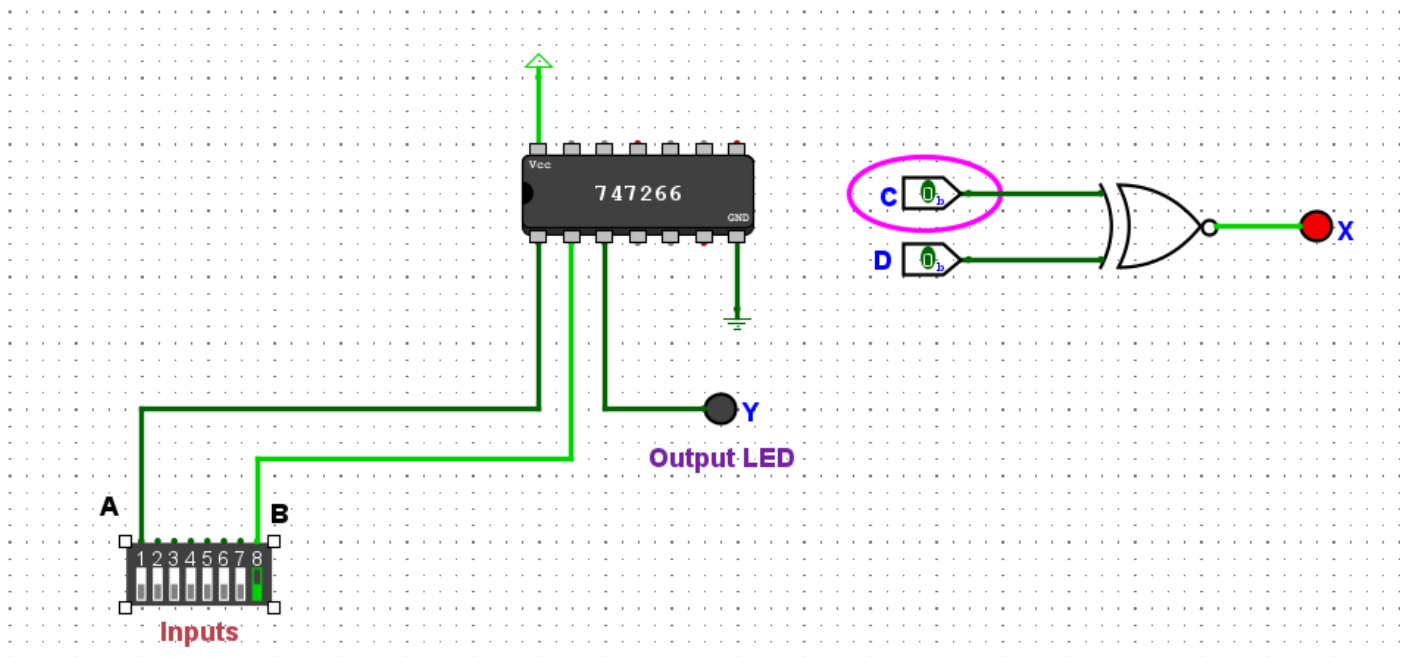


Figure 7: XNOR Gate (747266) Connect diagram with Truth Table

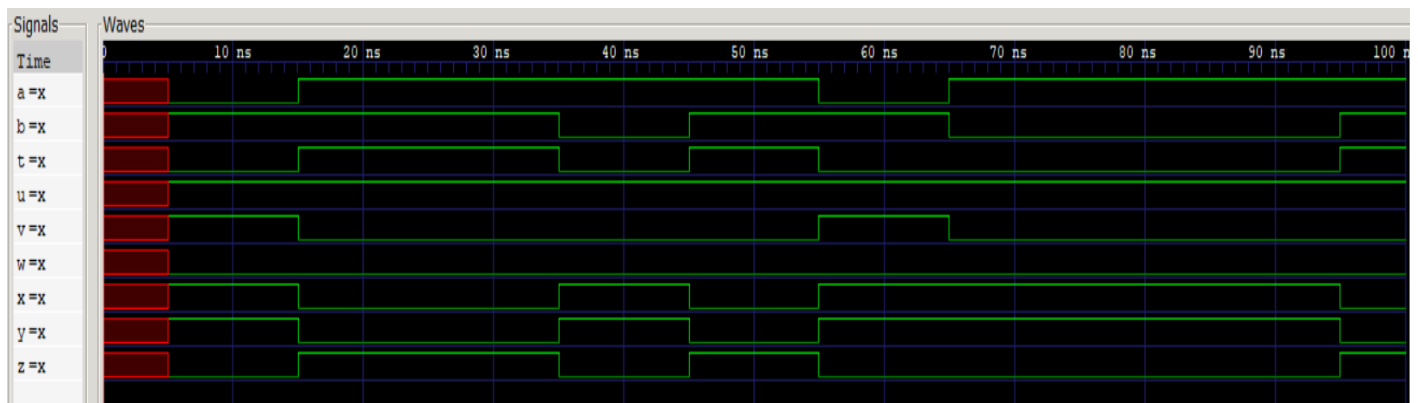


Figure 8: Basic Logic Gate Operations

The following Verilog assignments illustrate the fundamental behavior of digital logic gates using two input variables, a and b. The signal t represents the **AND** operation ($a \& b$), producing a high output only when both inputs are high. The signal u corresponds to the **OR** operation ($a \mid b$), which outputs high if either input is high. The signal v is the **inversion** of a ($\sim a$), while w performs the **NOR** operation ($\sim(a \mid b)$), yielding high only when both inputs are low. Similarly, x implements the **NAND** operation ($\sim(a \& b)$), which is low only when both inputs are high. The signal y denotes the **XOR** operation ($a \wedge b$), producing high when the inputs differ, and z represents the **XNOR** operation ($\sim(a \wedge b)$), which is high when the inputs are equal. Together, these assignments demonstrate the essential logic functions and their inverted forms, forming the basis of digital circuit design and verification.

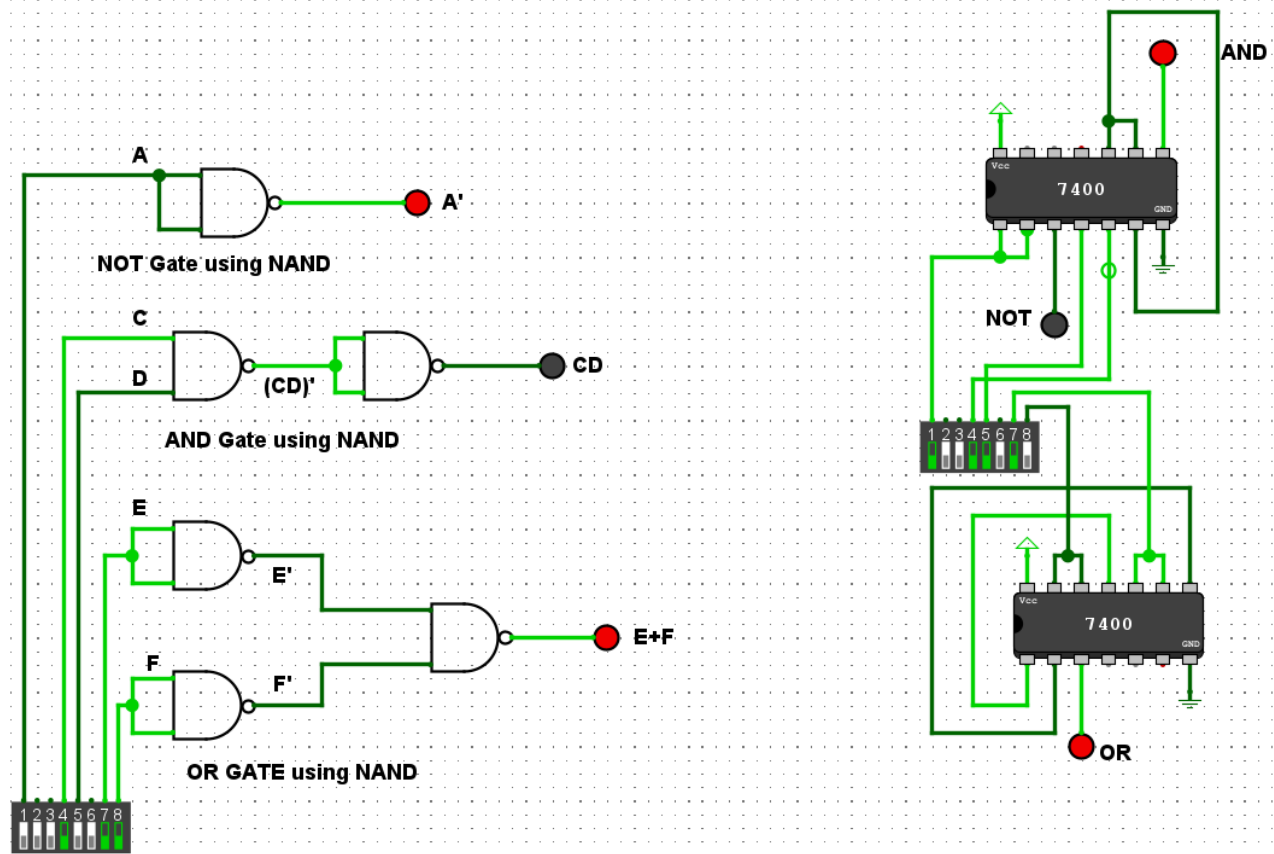


Figure 9: Universal Gate Implementation Using NAND Gates

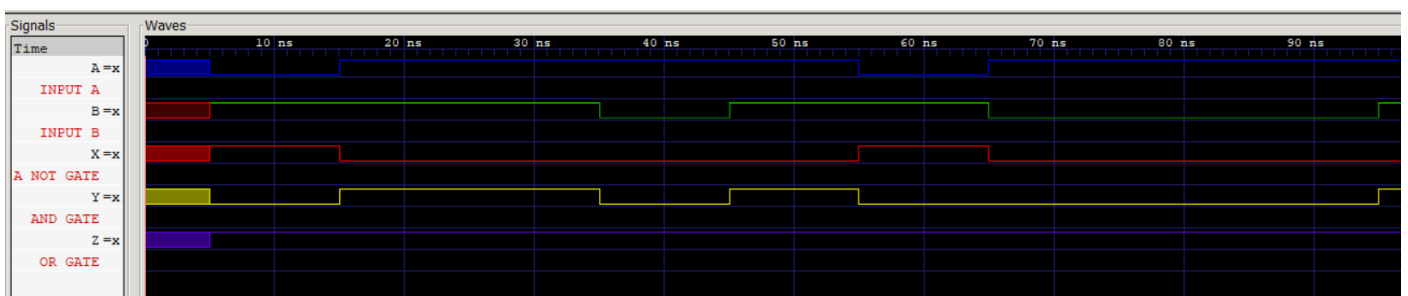


Figure 10: Verilog Simulation of NAND-Based Universal Logic

```

1 module nand_gate(
2     input A,
3     input B,
4     output X,Y,Z
5 );
6 wire u,v,w;
7
8 assign X = ~(A & A); //Not gate using NAND
9
10 assign u = ~(A & B); //AND gate using NAND
11 assign Y = ~(u & u);
12
13 assign v = ~(A & A); //OR gate using NAND
14 assign w = ~(B & B);
15 assign Z = ~(v & w);
16
17 endmodule

```

```

1 `timescale 1ns / 1ps
2
3 module nand_gate_tb;
4
5 // Inputs
6 reg A, B;
7 // Outputs
8 wire X, Y, Z;
9
10 // Instantiate your module
11 nand_gate uut (
12     .A(A),
13     .B(B),
14     .X(X),
15     .Y(Y),
16     .Z(Z)
17 );
18
19 // Clock Signal
20 reg clk;
21 initial clk = 0;
22 always #5 clk = ~clk; // 10 ns period
23
24 // Waveform dump
25 initial begin
26     $dumpfile("waveform.vcd"); // FIXED: it was $dupfile -> typo
27     $dumpvars(0, nand_gate_tb); // FIXED: it was $dupvars -> typo
28 end
29
30 // Generate random signals on clock
31 always @(posedge clk) begin
32     A = $random & 1; // mask to 0 or 1
33     B = $random & 1;
34 end
35
36 // Stop simulation after 100 ns
37 initial begin
38     #100 $finish;
39 end
40
41 endmodule

```

Figure 11: Verilog Code

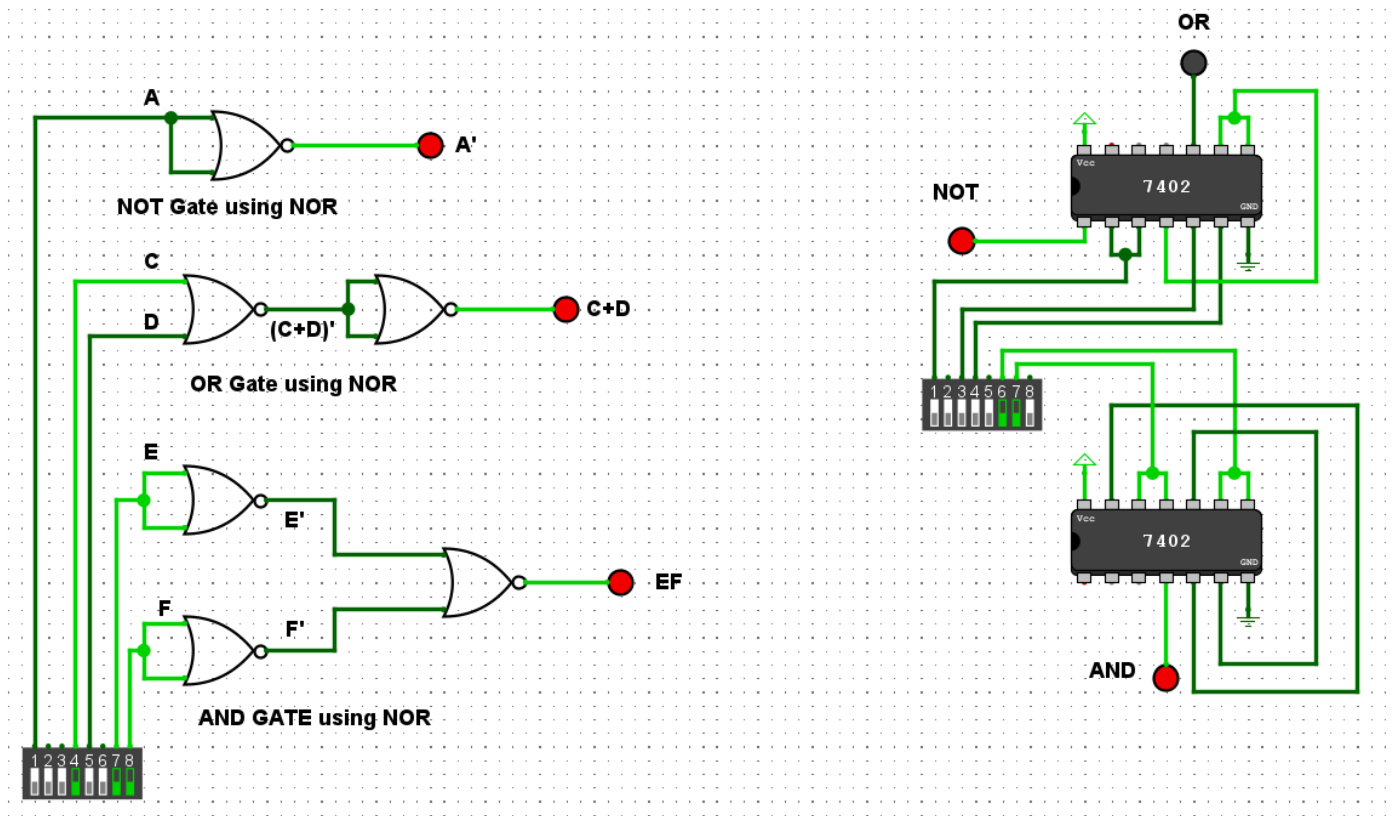


Figure 12: Universal Gate Implementation Using NOR Gates

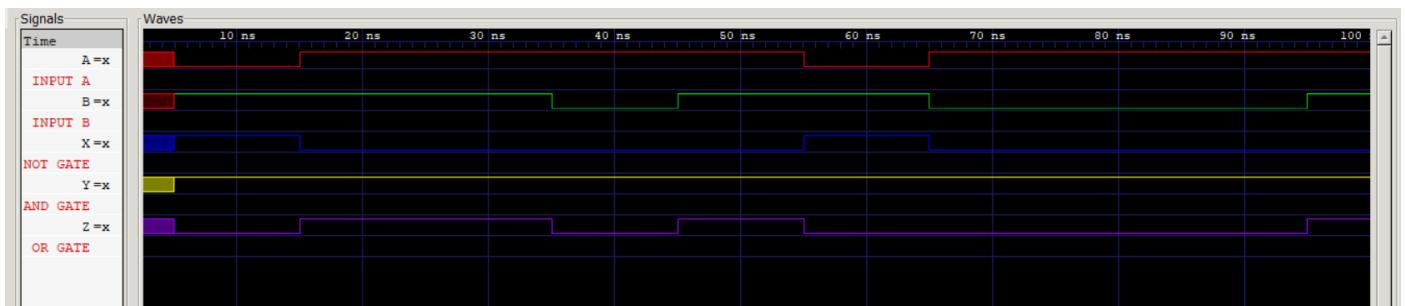


Figure 13: Figure 10: Verilog Simulation of NOR-Based Universal Logic

```

1 module nor_gate (
2     input A,
3     input B,
4     output X,
5     output Y,
6     output Z
7 );
8 wire a,b,c;
9
10 assign X = ~(A | A); // NOT gate
11
12 assign a = ~(A | B);
13 assign Y = ~(a | a); // AND gate
14
15 assign b = ~(A | A);
16 assign c = ~(B | B);
17 assign Z = ~(b | c); // OR gate
18
19 endmodule

```

```

1 `timescale 1ns / 1ps
2 module nor_gate_tb;
3
4     reg A,B;
5     wire X,Y,Z;
6
7     nor_gate uut (
8         .A(A),
9         .B(B),
10        .X(X),
11        .Y(Y),
12        .Z(Z)
13    );
14    initial begin
15        $dumpfile("wave.vcd");
16        $dumpvars(0, nor_gate_tb);
17    end
18
19    reg clk;
20    initial clk = 0;
21    always #5 clk = ~clk;
22
23    always @(posedge clk) begin
24        A <= $random & 1'b1;
25        B <= $random & 1'b1;
26    end
27
28    initial begin
29        #100 $finish;
30    end
31
32 endmodule

```

Figure 14: Verilog Code